

Informe de Seguridad — AuraDash

Cómo proteger la aplicación: qué hace la IA (el 80%) y qué hacés vos (el 20%)

Preparado para Vic · Suite de gestión para salones de estética

1. Resumen ejecutivo

Tu sistema no parte de cero en seguridad. Ya tenés varias capas bien implementadas: cabeceras de protección (helmet), límites de peticiones (rate limiting), una lista de orígenes permitidos (CORS), cifrado de datos sensibles, permisos por rol (RBAC), registro de auditoría y aislamiento entre negocios por businessId aplicado de forma consistente. La base es sólida.

El problema no es la arquitectura, son tres cosas concretas que hoy dejan la puerta abierta. Una vez resueltas, el sistema queda en buen estado para empezar a operar con negocios reales.

Las 3 cosas críticas

- 1) Contraseña de base de datos publicada.** El archivo DOCUMENTACION_SISTEMA.md está versionado en git e incluye la contraseña de la base (Neon) y un token de administrador. Cualquiera con acceso al repositorio o a su historial los tiene.
- 2) Token de admin “maestro” quemado en el código.** En requireAuth.js el token de carga rápida tiene un valor por defecto fijo, el mismo que aparece en la documentación. Quien lo conozca entra como dueño de un negocio sin login.
- 3) La autenticación “falla abierta”.** Si Firebase no logra verificar el token, el código actual concede acceso de administrador en vez de rechazar. Debe ser al revés: ante la duda, denegar.

2. El 80% — Lo que hace la IA (Antigravity)

Todo lo siguiente es código y configuración: se puede delegar. Copiá el prompt de abajo y pegalo en Antigravity (o pedímelo a mí) para que aplique los arreglos sobre tu repositorio.

Importante: antes de correr este prompt, rotá la contraseña de la base (ver Paso 1 de la sección 3). La IA limpia el código, pero la contraseña vieja solo la podés cambiar vos.

Prompt listo para copiar y pegar en Antigravity

CONTEXTO DEL PROYECTO

- Backend: Node.js + Express + Prisma (ORM) + Firebase Admin para autenticación. Carpeta backend/.
- Frontend: React + Vite en dashboard-react/.
- Base de datos: PostgreSQL (Neon). Backend en Render, frontend en Vercel.
- Es un SaaS multi-tenant: varios negocios (salones) en una misma base, aislados por businessId.

OBJETIVO

Endurecer la seguridad SIN romper las funciones que ya existen. Trabaja con cambios mínimos y explica cada cambio. NO inventes, no muestres ni escribas contraseñas ni tokens en el código: todo secreto debe leerse de variables de entorno.

TAREAS (en orden de prioridad)

- 1) Cerrar bypasses de autenticación en backend/src/middleware/requireAuth.js:
 - Elimina el valor por defecto del token de bypass (hoy "aura-admin-token"). El token de carga rápida debe leerse SOLO de process.env.QUICK_BOOKING_TOKEN. Si la variable no está definida, el bypass queda DESACTIVADO (sin ningún default).
 - Cuando ese token se use, NO otorgues admin:true ni rol owner: asigna un rol mínimo limitado únicamente a crear reservas.
 - Fail-closed: si verifyIdToken falla, devolvé SIEMPRE 401. Elimina el fallback que hoy concede acceso "quick-booking-user" cuando Firebase falla.
 - Los mecanismos de desarrollo AUTH_DISABLED===true y los tokens "local-token-..." deben ignorarse por completo cuando NODE_ENV===production.
- 2) Quitar el superadmin hardcodeado:
 - En backend/src/middleware/requireAdmin.js y backend/src/middleware/rbac.middleware.js, elimina el email fijo. Usa una lista leída de process.env.SUPERADMIN_EMAILS (separada por comas) o, mejor, un campo isSuperAdmin en el modelo User.
- 3) Endurecer CORS en backend/src/config/cors.js:
 - Elimina la regla comodín que acepta cualquier origen *.vercel.app y las redes privadas (192.168/172/10) cuando NODE_ENV===production.
 - Usa una allowlist explícita leída de process.env.FRONTEND_ORIGINS (separada por comas).
 - Mantén credentials:true solo con la allowlist exacta.
- 4) Validación de entrada con zod en TODOS los controladores que reciben body/params/query, priorizando public.controller.js, appointments.controller.js, clients.controller.js, finances.controller.js y billing.controller.js. Crea un middleware reusable validate(schema) y aplícalo en las rutas. Rechaza con 400 y mensaje genérico si no valida.
- 5) Aislamiento multi-tenant defensivo:
 - Agrega una verificación central que rechace toda petición autenticada que llegue a un controlador de negocio sin req.businessId.
 - (Avanzado, si es viable) habilita Row-Level Security en PostgreSQL por businessId como segunda barrera, además del filtro en la app.
- 6) Higiene de secretos:
 - Verifica que no haya secretos en el código ni en archivos versionados. El archivo DOCUMENTACION_SISTEMA.md contiene una contraseña y un token: móvelo fuera del repo o reemplaza esos valores por placeholders. Revisa el .gitignore.
 - Actualiza backend/.env.example con TODAS las variables nuevas (QUICK_BOOKING_TOKEN, SUPERADMIN_EMAILS, FRONTEND_ORIGINS) usando placeholders, sin valores reales.
- 7) Errores y logs:
 - Los mensajes de error devueltos al cliente deben ser genéricos (sin stack traces, sin SQL, sin rutas internas). El detalle se loguea solo en el servidor.

ENTREGABLES

- Lista de archivos modificados, con una explicación breve de cada cambio.
- Confirmación de que la app sigue compilando y que no quedó ningún secreto hardcodeado.
- Un README corto con las variables de entorno a configurar y su formato.

RESTRICCIONES

- No subas, no muestres ni inventes valores de secretos.

- No rompas el login real de Firebase ni el flujo de reservas públicas que ya funciona.

3. El 20% — Lo que hacés vos, paso a paso

Esto no es código: son acciones y decisiones tuyas. Ninguna IA puede hacerlas en tu lugar porque tocan secretos, infraestructura y temas legales. Están en orden; hacelas de arriba hacia abajo.

Paso 1. Cambiar la contraseña de la base de datos (Neon). Es lo más urgente.

- Entrá a neon.tech e iniciá sesión.
- Abrió tu proyecto → sección de conexión / roles → “Reset password” (restablecer contraseña).
- Copiá el NUEVO texto de conexión. Usá el que dice “Pooled connection” (termina en -pooler).

Paso 2. Actualizar esa contraseña en Render.

- Entrá a render.com → tu servicio del backend → pestaña “Environment”.
- Editá la variable `DATABASE_URL` y pegá el nuevo texto de conexión. Guardá: Render se vuelve a desplegar solo.

Paso 3. Cargar las variables nuevas en Render. En la misma pantalla “Environment”, agregá:

- `QUICK_BOOKING_TOKEN` = una cadena larga y al azar (mínimo 30 caracteres). Guardala en un lugar seguro; la vas a necesitar para el enlace de carga rápida.
- `SUPERADMIN_EMAILS` = tu email.
- `FRONTEND_ORIGINS` = tus dominios reales, ej.:
`https://auradash.digital,https://www.auradash.digital`
- `NODE_ENV` = production (verificá que ya esté).

Paso 4. Verificar el frontend (Vercel).

- Entrá a vercel.com → tu proyecto → Settings → Environment Variables.
- Confirmá que `VITE_AUTH_DISABLED` = false.
- Si cambiaste el token de carga rápida, actualizá el enlace y el código QR de carga rápida con el token nuevo.

Paso 5. Pasar el prompt a Antigravity. Copiá el prompt de la sección 2 y dejá que aplique los arreglos. Revisá la lista de cambios que te devuelva.

Paso 6. Sacar el archivo con secretos del repositorio.

- Borrá (o vaciá de datos sensibles) el archivo `DOCUMENTACION_SISTEMA.md`.
- Como ya rotaste la contraseña en el Paso 1, la vieja queda inservible aunque siga en el historial de git. (Borrarla también del historial es un paso avanzado opcional; con rotarla alcanza para neutralizar el riesgo.)
- Si el repositorio es público, ponelo privado.

Paso 7. Revisar lo legal de los datos de salud.

- Tus fichas guardan alergias y notas médicas. Antes de tener clientas reales, averiguá las obligaciones de protección de datos de tu país (consentimiento, derecho a borrado).
Conviene una consulta con alguien que sepa del tema; no es algo que resuelva el código.

Paso 8. **Revisión de seguridad final antes de cobrar.**

- Antes de salir a vender, conseguí una revisión de seguridad externa (un pentest básico).
La IA arregla el código; una mirada humana confirma que quedó bien.

4. Checklist final (en orden)

1. ☐ Rotar la contraseña de Neon — **vos**
2. ☐ Actualizar DATABASE_URL en Render — **vos**
3. ☐ Cargar variables nuevas en Render — **vos**
4. ☐ Verificar AUTH_DISABLED=false en Vercel — **vos**
5. ☐ Pasar el prompt a Antigravity — **IA hace los fixes**
6. ☐ Sacar el archivo con secretos del repo — **vos**
7. ☐ Revisar lo legal de datos de salud — **vos**
8. ☐ Revisión de seguridad externa antes de cobrar — **vos**

En una frase: la IA te deja el sistema técnicamente blindado; vos te encargás de las llaves, lo legal y la validación final. Si la IA hace su parte pero no rotás la contraseña, el sistema sigue inseguro — por eso ese 20% no es opcional.